1 Úloha 1: JPEG komprese a dekomprese rastru.

1.1 Zadání

Úkolem bylo implementovat algoritmus pro JPEG kompresi a rastru v prostředí MATLAB, zahrnující transformaci do YCbCr modelu, diskrétní kosinovou transformaci a kvantizaci koeficientů. Podmínkou bylo provést tyto kroky bez využití vestavěných funkcí. Součásti úkolu bylo také resamplování rastru, konverze pixelů do ZIG-ZAG sekvencí, následné Huffmanovo kódování, a dvě varianty náhrady DCT, a to s využitím diskrétní Fourierove transformace DFT a s využitím diskrétní vlnkové transformace DWT.

Algoritmy byly posuzovány na rastrech `gradient.bmp`, `mono.bmp`, `photo.bmp` a `vector.bmp` pro hodnoty faktoru $q = 10$, 50 a 70. Součásti vyhodnocení bylo provedeno vizuální porovnání výsledných rastrů, a také byly spočítany střední kvadratické odchylky jednotlivých složek RGB pro jednotlivé q.

1.2 Základní informace

Zpracovatelé: *Martin Ludvík, Maiia Liashenko*
Datum zpracování: *23.10.2026*
Prostředí zpracování: MATLAB; LaTeX
Použité AI nástroje: *ChatGPT, Claude.ai*

2 Základní část úlohy

2.1 Zadání

Úkolem bylo implementovat v prostředí MATLAB základní fáze ztrátové komprese obrazu založený na diskrétní kosinové transformaci (DCT). Zpracování zahrnovalo načtení rastrového obrazu, převod barevného prostoru RGB do YCbCr, rozdělení obrazu na bloky 8×8, výpočet DCT, kvantizaci transformačních koeficientů a následnou rekonstrukci pomocí dekvantizace a inverzní DCT. Rekonstruovaný obraz byl převeden zpět do RGB a byl porovnán s originálem prostřednictvím střední kvadratické odchylky m.

2.2 Popis a rozbor problému

Princip a pořadí zpracování. Nejprve byly prostorové hodnoty pixelů převedeny do barevného prostoru YCbCr, ve kterém je jasová informace oddělena od chromatické. Každá složka byla následně zpracována po blocích 8×8. Diskrétní kosinovou transformací byly převedeny hodnoty pixelů na koeficienty prostorových frekvencí. Následnou kvantizaci koeficienty s malou hodnotou byly vypuštěny. Při rekonstrukci se operace provádějí v opačném pořadí, původní hodnoty již však nelze po kvantizaci získat beze ztráty.

Převod barevného prostoru RGB do YCbCr. Vstupní barevný rastr obsahoval složky R, G a B s hodnotami jednotlivých pixelů. Pro další zpracování byly vytvořeny složky Y (jasová), Cb (modrá chrominační) a Cr (červená chrominační). V použitém algoritmu byl převod definován následujícími vztahy:

$$Y = 0{,}2990\,R + 0{,}5870\,G + 0{,}1140\,B, \tag{1}$$

$$Cb = -0{,}1687\,R - 0{,}3313\,G + 0{,}5000\,B + 128, \tag{2}$$

$$Cr = 0{,}5000\,R - 0{,}4187\,G - 0{,}0813\,B + 128. \tag{3}$$

Cílem tohoto kroku bylo co nejmenší poškození jasové složky Y.

Diskrétní kosinová transformace. Pro blok velikosti 8×8 byl použit vztah:

$$F(u,v) = \frac{1}{4}\, c(u)c(v) \sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y) \cos\left(\frac{(2x+1)u\pi}{16}\right) \cos\left(\frac{(2y+1)v\pi}{16}\right), \tag{4}$$

kde $f(x,y)$ je původní hodnotou pixelu v bloku, $F(u,v)$ transformační koeficient a $u, v \in \{0,\ldots,7\}$ indexy prostorových frekvencí. Normalizační faktory jsou

$$c(t) = \begin{cases} \dfrac{1}{\sqrt{2}}, & t = 0, \\ 1, & t > 0. \end{cases} \tag{5}$$

Koeficient $F(0,0)$ je označován jako *DC člen*, je největší hodnotou matici F a má největší vliv na kvalitu obrazu. Ostatní koeficienty (*AC členy*) popisují změny hodnot pixelů s různými prostorovými frekvencemi. Prvky s vyššími hodnotami jsou soustředěny v levém horním rohu matice F.

Kvantizace koeficientů. Kvantizace je hlavní ztrátovou operací základního algoritmu. Každý koeficient DCT je vydělen příslušnou hodnotou kvantizační matice a zaokrouhlen na celé číslo:

$$F_Q(u,v) = \operatorname{round}\left(\frac{F(u,v)}{Q_q(u,v)}\right). \tag{6}$$

V algoritmu byly použity dvě výchozí matice: Q_Y pro jasovou složku a Q_C pro obě chromatické složky. Tento krok byl proveden za účelem vypuštění koeficientů s malou hodnotou.

Matice s kvantizovanými koeficienty byla spočítana podle následujícího vztahu:

$$Q_q = \frac{50}{q}\, Q_{50}. \tag{7}$$

Dekvantizace a inverzní DCT. Při rekonstrukci byly kvantizované koeficienty nejprve vynásobí odpovídající kvantizační maticí:

$$\widetilde{F}(u,v) = F_Q(u,v)\, Q_q(u,v). \tag{8}$$

Inverzní DCT byla spočítana podle vztahu:

$$\widetilde{f}(x,y) = \frac{1}{4} \sum_{u=0}^{7} \sum_{v=0}^{7} c(u)c(v)\, \widetilde{F}(u,v) \cos\left(\frac{(2x+1)u\pi}{16}\right) \cos\left(\frac{(2y+1)v\pi}{16}\right). \tag{9}$$

Rekonstruované bloky byly uloženy na odpovídající pozice v maticích Y, Cb a Cr. Převod zpět do RGB byl realizován těmito vztahy:

$$\widetilde{R} = \widetilde{Y} + 1{,}4020\,(\widetilde{Cr} - 128), \tag{10}$$

$$\widetilde{G} = \widetilde{Y} - 0{,}3441\,(\widetilde{Cb} - 128) - 0{,}7141\,(\widetilde{Cr} - 128), \tag{11}$$

$$\widetilde{B} = \widetilde{Y} + 1{,}7720\,(\widetilde{Cb} - 128) - 0{,}0001\,(\widetilde{Cr} - 128). \tag{12}$$

Barevné složky byly spojeny do výsledného RGB rastru.

Posouzení přesnosti rekonstrukce. Kvalita rekonstruovaného obrazu byla vyhodnocena zvlášť pro každou složku RGB pomocí odmocniny ze střední kvadratické chyby:

$$m_C = \sqrt{\frac{1}{mn} \sum_{i=1}^{m} \sum_{j=1}^{n} \left(C_{\text{orig}}(i,j) - C_{\text{rek}}(i,j)\right)^2}, \qquad C \in \{R,G,B\}. \tag{13}$$

Čím menší byla výsledná odchylka, tím blíže byla rekonstruovaná složka původnímu obrazu.

2.3 Popis algoritmů

Načtení rastru a převod barevného prostoru. Příkaz `imread` načetl obrazový soubor BMP. Z trojrozměrného pole se pomocí `raster(:,:,1)`, `raster(:,:,2)` a `raster(:,:,3)` byly získany složky R, G a B. Následně se podle výše uvedených lineárních vztahů byly vypočtěny matice `Y`, `Cb` a `Cr`. Do DCT vstupovaly tyto tři matice přímo, bez jejich prostorového převzorkování.

Funkce `mydct(R,Q)` – rozdělení rastru a kvantizace.

1. Pomocí `size(R)` se zjistily rozměry zpracovávané komponenty a vytvořily se výstupní matice `Rt`.
2. Dva vnořené cykly s krokem osm procházely obraz po nepřekrývajících se blocích 8×8 a vybíraly je příkazem `R(i:i+7,j:j+7)`.
3. Na každý blok byla zavolana funkce `mydctsub`, která vypočítávala 64 transformačních koeficientů.
4. Koeficienty byly vyděleny po prvcích kvantizační maticí `Q` (operátor `./`) a byly zaokrouhleny příkazem `round`.
5. Kvantizovaný blok byl uložen na odpovídající pozici do výstupní matice `Rt`.

Funkce `mydctsub(R)` – výpočet DCT bloku 8×8. Vnější cykly procházely indexy výstupních koeficientů $u, v = 0, \ldots, 7$. Podle hodnot u a v byly nastaveny normalizační koeficienty `cu` a `cv`. Pomocná proměnná `F` se pro každý koeficient vynulovala a vnitřní dvojice cyklů přes x a y sčítavala příspěvky všech 64 pixelů podle vztahu (4). Výsledek byl uložen do `Rt(u+1,v+1)`.

Funkce `myidct(R,Q)` – dekvantizace a rekonstrukce rastru. Funkce procházela matici kvantizovaných koeficientů v blocích 8×8. V každém bloku se provaděla dekvantizace operací `R_sub .* Q` a poté se zavolala `myidctsub`. Výsledná matice prostorových hodnot nahrazovala původní kvantizovaný blok na stejné pozici výstupu.

Funkce `myidctsub(R)` – inverzní DCT bloku 8×8. Dvojice vnějších cyklů postupně určovala polohu výsledného pixelu x, y. Pro každý pixel byl vynulovan `F`. Vnitřní cykly procházely všech 64 koeficientů u, v. Získaná hodnota byla zapsana do `Rt(x+1,y+1)`.

Převod zpět do RGB a funkce `comp_er(c1,c2)`. Po inverzní DCT byly spočítany rekonstruované složky `R2`, `G2` a `B2`. Spojením do `raster2(:,:,1:3)` vznikl barevný rastr. Funkce `comp_er` vracela střední hodnotu po odmocnění podle vztahu (13).

2.4 Závěr

Byl popsán a implementován základní algoritmus ztrátové obrazové komprese založený na DCT. Se zvětšující se hodnotou q dochazelo k menší ztrátě detailů a nižším hodnotam m, konkrétní velikost odchylek však závisí na zpracovaném obrazu. Grafické porovnání a znázornění hodnot středních kvadratických odchylek m jsou k nahlednutí v kapitole č.3, kde jsou tyto výsledky porovnány s výsledky po použití komprese založených na DFT s DWT.

3 Převzorkování, ZIGZAG sekvence a Huffmanovo kódování

3.1 Zadání

Úkolem bylo doplnit algoritmus komprese JPEG o tři operace: převzorkování rastrových dat, převod kvantizovaných transformačních koeficientů do ZIGZAG sekvence a jejich bezeztrátové Huffmanovo kódování. Součástí řešení bylo také dekódování a obnovení původního uspořádání koeficientů pro následnou inverzní transformaci.

3.2 Popis a rozbor problému

Převzorkování. Převzorkování bylo provedeno průměrováním submatic 2x2 pixely. Cílem resamplování bylo snížení počtu barevných odstínů. Pro každou submatici 2×2 se čtyři hodnoty nahradí jediným průměrem:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \quad \longrightarrow \quad \frac{10+20+30+40}{4} = 25. \tag{14}$$

ZIGZAG sekvence. Pomocí ZIGZAG metody byla převedena dvourozměrná matice po DCT kvantizaci na jednorozměrný vektor. Výhodou této metody bylo vytvoření takové posloupnosti koeficientů, kterou bylo možné efektivně komprimovat Huffmanovým kódováním. Průchod mytici byl určen součtem indexů

$$s = i + j, \qquad 2 \leq s \leq n + m, \tag{15}$$

kde i označuje řádek a j sloupec. Pro sudé s se daná diagonála procházela nahoru doprava, pro liché s dolů doleva. Rozsah přípustných řádkových indexů je

$$\max(1, s-m) \leq i \leq \min(n, s-1), \qquad j = s - i. \tag{16}$$

Samotné přeuspořádání koeficientů bylo bezeztrátové a provádělo se za účelem efektivnějšího zakódování.

Huffmanovo kódování. Huffmanovo kódování je založeno na principu přiřazení symbolům binárních kódů podle četnosti jejich výskytu. Nejprve byla sestavena tabulka unikátních hodnot a jejich četností. Dva uzly s nejnižší četností byly spojeny do nového rodičovského uzlu, jehož četnost se rovnála součtu četností obou potomků. Postup se opakoval, dokud nevznikl jediný kořenový uzel. Levým větvím stromu byl přiřazen bit 0, pravým bit 1.

Symbol	Četnost	Příklad Huffmanova kódu
1	5	`0`
2	3	`10`
3	2	`110`
4	1	`1110`
5	1	`1111`

Jde o ilustrační příklad. Při dekompresi byl použít stejný slovník jako při kompresi.

3.3 Popis algoritmů

Převzorkování: `resamp_22`.

1. Určení počtu řádků a sloupců vstupní matice a výstupních rozměrů.
2. Vytvoření nulové výstupní matice příkazem `zeros`.
3. Průchod vstupní maticí pomocí dvou vnořených cyklů s krokem 2.

4. Výpočet průměru vybrané submatice příkazem `mean(cellR(:))`.
5. Uložení průměru na odpovídající pozici ve výstupní matici.

ZIGZAG: `zigzag_scan` a `zigzag_blocks`.

1. Funkce `zigzag_scan` inicializovala výstupní vektor a procházela diagonály určené součtem $s = i + j$.
2. Podle s se měnil směr procházení diagonály. Aktuální koeficient se ukládal do další pozice vektoru.
3. Funkce `zigzag_blocks` rozdělovala matici po blocích 8×8 a postupně volala `zigzag_scan`.
4. Vektory jednotlivých bloků se spojovaly do jedné sloupcové posloupnosti.

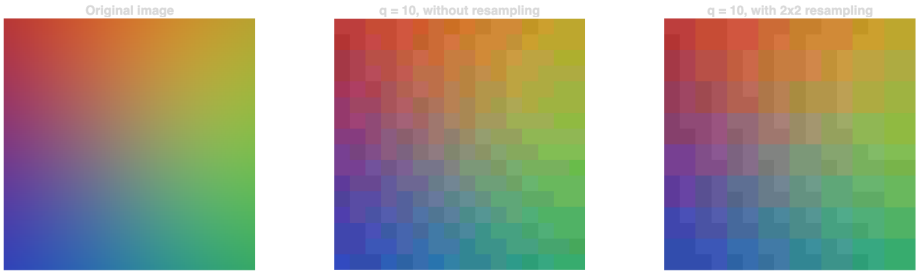
Huffman: `huffman_compres` a `huffman_decompress`.

1. Funkce `unique` určovala unikátní hodnoty a `accumarray` spočítávala jejich četnosti.
2. Četnosti se seřazovaly a inicializovaly se matice `tree` se sloupci *ID uzlu, četnost, levý potomek, pravý potomek*.
3. V cyklu se opakovaně spojovaly dva aktivní uzly s nejmenší četností. Nový uzel se vracel do seznamu aktivních uzlů.
4. Z dokončeného stromu pro každý původní symbol byla určena cesta od listu ke kořeni. Bity 0 a 1 se při tomto opačném průchodu vkládaly na začátek vytvářeného kódu.
5. Dvojice byla uložena do slovníku `huffman_dict`. Každá hodnota vstupní posloupnosti byla potom nahrazena svým kódem.
6. Při dekódování byly bity postupně připojeny k pomocnému řetězci. Funkce `strcmp` vyhledavala úplný kód ve slovníku. Po nalezení se ukládala příslušná původní hodnota a pomocný řetězec se vyprázdňoval.

3.4 Obrazové porovnání převzorkování

Pro všechny čtyři vstupní rastry (`gradient.bmp`, `mono.bmp`, `photo.bmp` a `vector.bmp`) byly uvažovány hodnoty $q = 10$, $q = 50$ a $q = 70$.

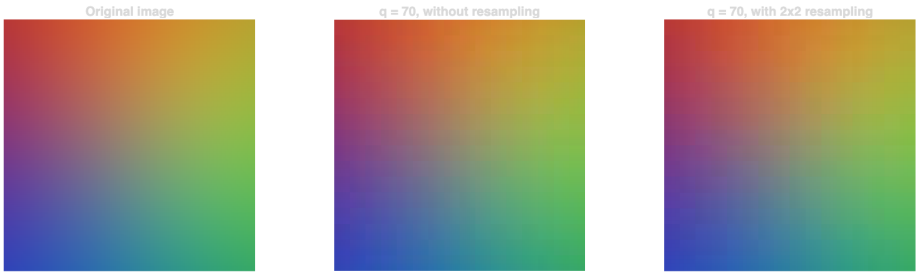
ZIGZAG sekvence a Huffmanovo kódování nebyly do obrazového porovnání zahrnuty, protože šlo o bezeztrátové operace.



(a) $q = 10$



(b) $q = 50$



(c) $q = 70$

Obrázek 1: Barevný gradient: originál, rekonstrukce bez převzorkování a rekonstrukce s převzorkováním 2×2 pro tři hodnoty q.



(a) $q = 10$

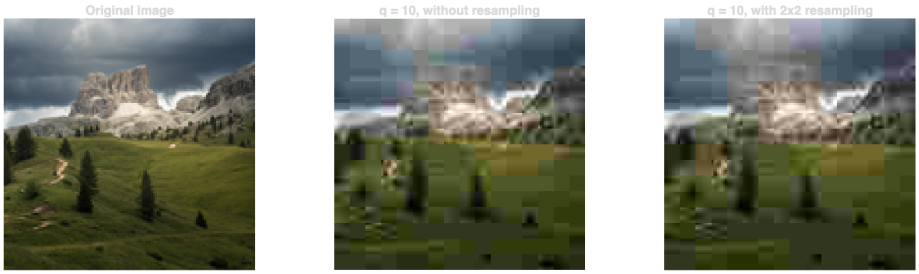


(b) $q = 50$

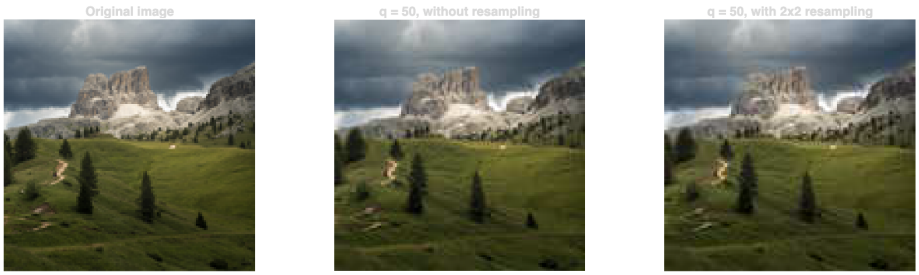


(c) $q = 70$

Obrázek 2: Černobílý gradient: originál, rekonstrukce bez převzorkování a rekonstrukce s převzorkováním 2×2 pro tři hodnoty q.



(a) $q = 10$



(b) $q = 50$

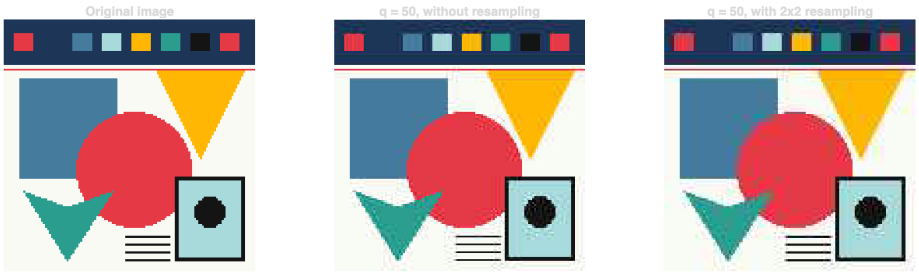


(c) $q = 70$

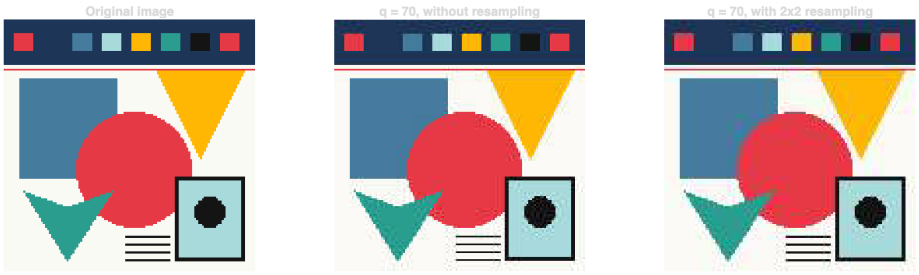
Obrázek 3: Barevná fotografie: originál, rekonstrukce bez převzorkování a rekonstrukce s převzorkováním 2×2 pro tři hodnoty q.



(a) $q = 10$



(b) $q = 50$



(c) $q = 70$

Obrázek 4: Rastr z vektorových obrazců: originál, rekonstrukce bez převzorkování a rekonstrukce s převzorkováním 2×2 pro tři hodnoty q.

3.5 Závěr

Byly implementovány tři navazující části kompresního algoritmu JPEG. Převzorkování průměrováním submatic snižovalo počet chromatických vzorků, a tím množství dále zpracovávaných dat. ZIGZAG převod i Huffmanovo kódování jsou bezeztratové, jejich účinnost lze posoudit na základě délky zakódované posloupnosti.

		Bez převzorkování			Převzorkování 2×2		
Rastr	q	m_R	m_G	m_B	m_R	m_G	m_B
`gradient`	10	6.00	4.41	6.64	7.68	4.96	8.74
`gradient`	50	1.89	1.34	2.51	1.85	1.36	2.33
`gradient`	70	1.40	0.99	1.45	1.61	1.03	1.92
`mono`	10	6.79	6.87	6.80	6.79	6.87	6.80
`mono`	50	3.50	3.50	3.52	3.50	3.50	3.52
`mono`	70	3.14	3.12	3.18	3.14	3.12	3.18
`photo`	10	11.76	10.52	11.69	11.99	10.55	12.58
`photo`	50	6.43	5.82	6.43	6.66	5.86	7.28
`photo`	70	5.63	5.10	5.72	5.86	5.16	6.65
`vector`	10	19.46	15.37	19.51	24.88	16.46	24.11
`vector`	50	10.30	7.76	11.42	17.38	9.96	16.71
`vector`	70	8.76	6.60	9.13	15.92	9.12	15.13

Tabulka 1: Střední kvadratické odchylky (m; RMSE) jednotlivých složek RGB pro všechny rastry a hodnoty q.

		Bez převzorkování		Převzorkování 2×2	
Rastr	q	Bitů	CR_{16}	Bitů	CR_{16}
`gradient`	10	51777	15.19	25828	15.22
`gradient`	50	55720	14.11	28088	14.00
`gradient`	70	56705	13.87	28691	13.71
`mono`	10	51699	15.21	27123	14.50
`mono`	50	56867	13.83	32291	12.18
`mono`	70	58717	13.39	34141	11.52
`photo`	10	52911	14.86	27605	14.24
`photo`	50	62481	12.59	35684	11.02
`photo`	70	66059	11.90	38635	10.18
`vector`	10	56220	13.99	29894	13.15
`vector`	50	66534	11.82	37636	10.45
`vector`	70	69909	11.25	40035	9.82

Tabulka 2: Délka Huffmanových posloupností a orientační kompresní poměr CR_{16} vůči 16bitovému uložení kvantovaných koeficientů (bez režie slovníků).

4 Náhrada DCT pomocí DFT a DWT

4.1 Zadání

Cílem úloh bylo nahradit diskrétní kosinovou transformaci (DCT) v algoritmu JPEG diskrétní Fourierovou transformací (DFT) a diskrétní vlnkovou transformací (DWT) a výsledky porovnat s DCT. Obě varianty byly testovány na stejných rastrech a se stejnými faktory komprese $q = 10, 50, 70$ jako DCT.

Obě varianty používají stejný kompresní řetězec jako DCT včetně převzorkování, ZIG-ZAG sekvencí a Huffmanova kódování. Tyto funkce které byly použity pro všechny tři transformace

jsou uloženy v samostatných souborech.

4.2 Popis a rozbor problému

Všechny tři transformace rozkládají blok 8×8 na součet bázových funkcí. Samotná transformace je bezeztrátová, ke ztrátě dochází až kvantizací. Vhodnost transformace proto určuje, kolika koeficienty dokáže blok popsat: čím méně jich je, tím menší je chyba po kvantizaci [2].

DFT. Blok se rozkládá na komplexní exponenciály (dvojice kosinus a sinus). Normalizace $1/8$ dává koeficienty ve stejném měřítku jako DCT ($F(0,0) = 8\times$ průměr bloku):

$$F(u,v) = \frac{1}{8} \sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y)\, e^{-2\pi i\,(ux+vy)/8}, \qquad f(x,y) = \frac{1}{8} \sum_{u=0}^{7} \sum_{v=0}^{7} F(u,v)\, e^{2\pi i\,(ux+vy)/8}. \quad (17)$$

Koeficienty jsou komplexní a konjugovaně symetrické, $F(8-u, 8-v) = \overline{F(u,v)}$ (indexy modulo 8). Index k a $8-k$ odpovídají stejné frekvenci, takže nízké frekvence leží ve všech rozích matice. Hlavní nevýhodou je, že DFT předpokládá periodické opakování bloku: rozdílné hodnoty na protilehlých okrajích vytvoří umělý skok a energie se rozptýlí do vysokých frekvencí.

DWT (Haar). Haarova transformace nahrazuje dvojice sousedních hodnot jejich průměrem a rozdílem:

$$s_k = \frac{x_{2k-1} + x_{2k}}{\sqrt{2}}, \qquad d_k = \frac{x_{2k-1} - x_{2k}}{\sqrt{2}}, \qquad k = 1, \ldots, L/2. \quad (18)$$

Krok lze zapsat ortogonální maticí H_L (horní polovina řádků počítá průměry, dolní rozdíly). Pro blok 8×8 se provedou tři úrovně, každá vždy jen na pásmu průměrů v levém horním rohu:

$$S_{1:L,\,1:L} \leftarrow H_L\, S_{1:L,\,1:L}\, H_L^T, \qquad L = 8,\, 4,\, 2, \qquad H_L^{-1} = H_L^T. \quad (19)$$

Bázové funkce jsou nenulové jen v části bloku, takže skoková hrana ovlivní jen několik koeficientů. Lineární přechod naopak dá mnoho malých nenulových rozdílů.

	DCT	DFT	DWT (Haar)
Bázové funkce	kosiny	kosiny a siny	schodové funkce
Okraj bloku	zrcadlení (plynulé)	periodické opakování (skok)	lokální báze
Vhodná data	plynulé přechody	periodické vzory	ostré hrany
Nevhodná data	ostré hrany	běžná obrazová data	lineární přechody

4.3 Popis algoritmů

Kompresní řetězec je pro všechny transformace shodný: převod RGB → YCbCr, převzorkování Cb, Cr (2×2), posun hodnot o -128, rozdělení na bloky 8×8, transformace a kvantizace, ZIG-ZAG a Huffmanovo kódování. Dekomprese probíhá v opačném pořadí [4]. Kvantizační matice Q_{50} se pro faktor q škáluje vztahem

$$S = \begin{cases} 50/q, & q < 50, \\ 2 - q/50, & q \geq 50, \end{cases} \qquad Q_s = \max\left(1,\ \mathrm{round}(S \cdot Q_{50})\right). \tag{20}$$

Pro DFT a DWT se hodnoty Q_s pouze přeskládají podle polohy odpovídajících frekvencí, aby byla srovnání s DCT férová.

DFT.

1. Výpočet koeficientů podle (17).
2. Vynucení konjugované symetrie $F \leftarrow \frac{1}{2}\big(F + \overline{F(8-u,\, 8-v)}\big)$, aby se sdružené koeficienty nezaokrouhlily různě vlivem numerických chyb.
3. Kvantizace reálné a imaginární části zvlášť maticí $Q_{\mathrm{DFT}}(u,v) = Q_s\big(d(u), d(v)\big)$, kde $d(k) = \min\left(2\min(k, 8-k), 7\right)$ převádí index DFT na index DCT se stejnou frekvencí.
4. Kódování: do ZIG-ZAG a Huffmanova kódování vstupují reálné části následované imaginárními.

Při dekompresi se obě části složí, dekvantují, zpětně transformují a ponechá se reálná část výsledku.

DWT.

1. Tři úrovně Haarovy transformace podle (19) pro $L = 8, 4, 2$.
2. Kvantizace maticí Q_{DWT}, ve které má každé pásmo (pozice $\{1\}$, $\{2\}$, $\{3,4\}$, $\{5,\ldots,8\}$ v každé ose) jeden krok rovný průměru hodnot Q_s v tomto pásmu.
3. ZIG-ZAG a Huffmanovo kódování beze změny, protože koeficienty jsou reálné.

Při dekompresi se úrovně zpracují v obráceném pořadí operací $S \leftarrow H_L^T S H_L$.

4.4 Vstupní data

Čtyři rastry 128×128 px ve formátu BMP: barevná fotografie (`photo.bmp`), barevný gradient (`gradient.bmp`), černobílý gradient (`mono.bmp`) a rastr z vektorových obrazců s ostrými hranami (`vector.bmp`). Dalšími vstupy jsou faktory komprese q a matice Q_{50} podle ITU-T T.81 [1].

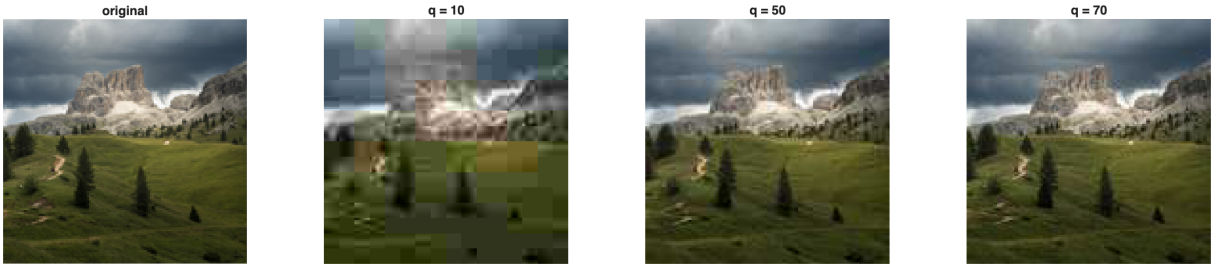
4.5 Výstupní data

Pro každý rastr se zobrazí originál a rekonstrukce pro všechna q. Do konzole se vypíší tabulky střední kvadratické odchylky m složek R, G, B a kompresního poměru CR (bez velikosti Huffmanových slovníků):

$$m = \sqrt{\frac{\sum_{i=1}^{m \cdot n} (z_i - z_i')^2}{m \cdot n}}, \qquad CR = \frac{8 \cdot 3 \cdot m \cdot n}{b_Y + b_{Cb} + b_{Cr}}, \tag{21}$$

kde b_Y, b_{Cb}, b_{Cr} jsou počty bitů Huffmanova kódu jednotlivých složek.

(a) DCT



(b) DWT

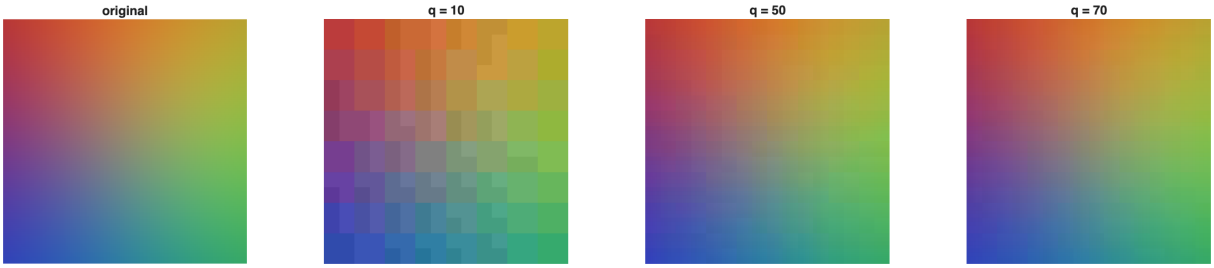


(c) DFT

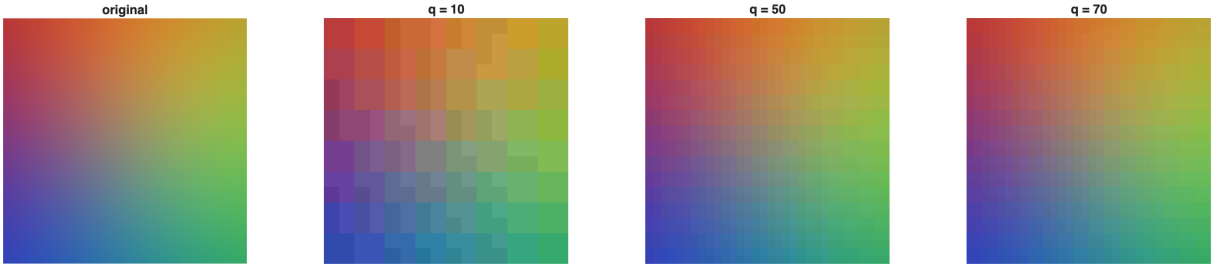


Obrázek 5: Barevná fotografie `photo.bmp`: originál a pro $q = 10$, 50 a 70.

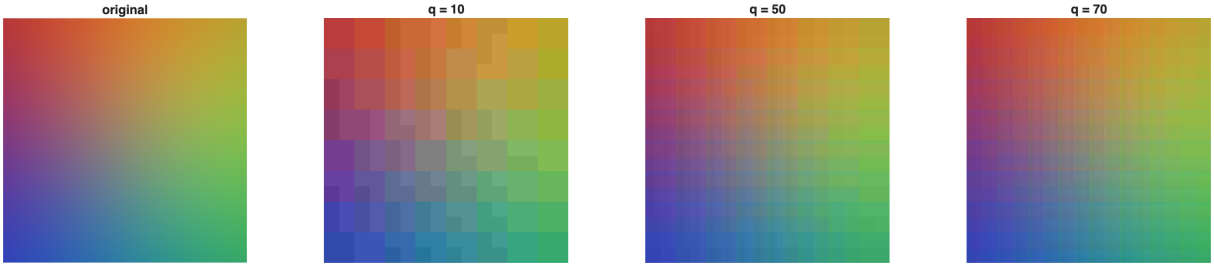
(a) DCT



(b) DWT

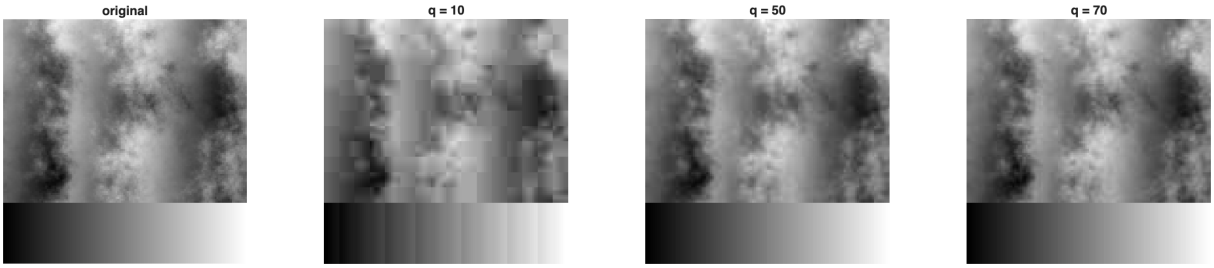


(c) DFT

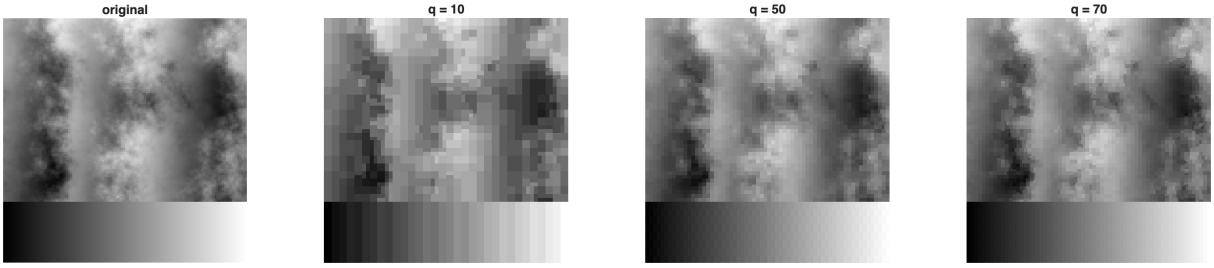


Obrázek 6: Barevný gradient `gradient.bmp`: originál a pro $q = 10$, 50 a 70.

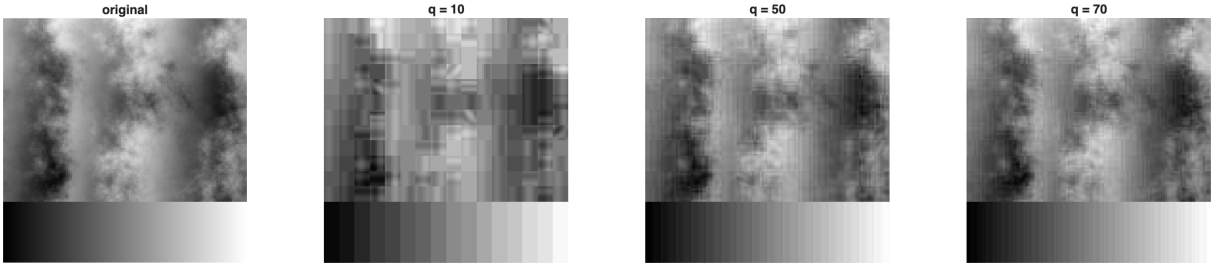
(a) DCT



(b) DWT

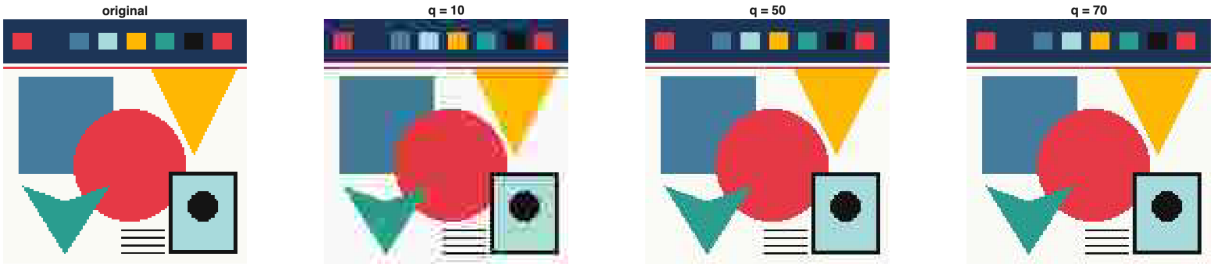


(c) DFT

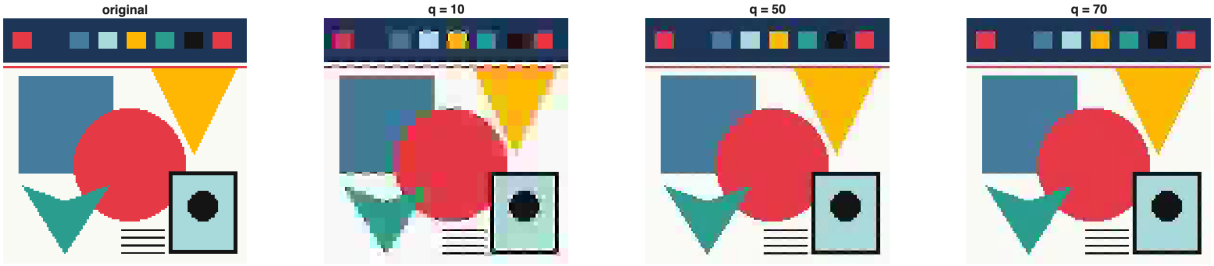


Obrázek 7: Černobílý gradient `mono.bmp`: originál a pro $q = 10$, 50 a 70.

(a) DCT



(b) DWT



(c) DFT



Obrázek 8: Rastr z vektorových obrazců `vector.bmp`: originál a pro $q = 10$, 50 a 70.

4.6 Závěr

Tabulka uvádí odchylky m_R / m_G / m_B (u `mono` platí $m_R = m_G = m_B$); nejmenší odchylka v řádku je zvýrazněna.

Rastr	q	DCT	DWT	DFT	CR DCT / DWT / DFT
`photo`	10	**11,87 / 10,62 / 12,58**	13,26 / 11,94 / 13,31	14,64 / 13,35 / 14,25	14,48 / 14,58 / 7,60
`photo`	50	**6,69 / 5,87 / 7,26**	8,00 / 7,21 / 8,30	9,25 / 8,30 / 9,47	11,19 / 11,39 / 6,35
`photo`	70	**5,50 / 4,83 / 6,36**	6,39 / 5,67 / 7,19	7,67 / 6,87 / 8,44	9,85 / 10,01 / 5,72
`gradient`	10	7,66 / 4,83 / 8,58	7,66 / 4,83 / 8,58	7,66 / 4,83 / 8,58	15,35 / 15,35 / 7,84
`gradient`	50	**1,83 / 1,36 / 2,30**	3,28 / 2,03 / 3,73	5,01 / 2,81 / 6,02	14,10 / 14,17 / 7,51
`gradient`	70	**1,46 / 0,96 / 1,73**	3,01 / 1,80 / 3,46	4,08 / 2,47 / 4,66	13,68 / 13,75 / 7,34
`mono`	10	**6,80**	8,29	10,15	14,70 / 14,71 / 7,67
`mono`	50	**3,44**	4,96	5,99	12,33 / 12,12 / 6,59
`mono`	70	**2,90**	4,29	4,92	11,27 / 10,97 / 6,05
`vector`	10	**25,00 / 16,54 / 23,99**	26,58 / 18,66 / 27,43	30,31 / 23,67 / 33,31	13,48 / 13,62 / 7,25
`vector`	50	17,40 / 9,98 / 16,70	**16,70 / 8,79 / 15,50**	21,18 / 13,07 / 20,47	10,60 / 11,26 / 6,01
`vector`	70	15,41 / 8,44 / 14,53	**14,17 / 7,72 / 13,68**	18,23 / 10,79 / 17,56	9,62 / 10,58 / 5,52

- **DCT** dává nejmenší chybu u fotografie a obou gradientů, tedy u dat s plynulými přechody.
- **DWT** je nejlepší u ostrých hran (`vector`) při $q = 50$ a 70, a to při vyšším kompresním poměru než DCT. Odpovídá to lokálnímu charakteru Haarových bázových funkcí a závěrům [3]. Při $q = 10$ hrubá kvantizace nahradí hrany schody a lepší je DCT.
- **DFT** je nejhorší ve všech případech kvůli umělým skokům na okrajích bloku. Kompresní poměr má zhruba poloviční, protože se kódují reálné i imaginární části koeficientů.
- U **barevného gradientu při** $q = 10$ dávají všechny transformace shodný výsledek: po hrubé kvantizaci zbude ve všech případech jen průměr bloku.

Neřešené problémy a náměty na vylepšení. Kódování DFT je redundantní, stačilo by kódovat jen nezávislou polovinu koeficientů. Kompresní poměr omezuje Huffmanovo kódování každého koeficientu alespoň jedním bitem; řešením je kódování běhů nul (RLE) jako ve standardu JPEG. DWT by bylo vhodné aplikovat na celý rastr místo bloků 8×8 a Haarovu vlnku nahradit vlnkou CDF 9/7 používanou v JPEG 2000. [AI]

4.7 Použití AI

Při vypracování bylo použito Claude.ai a to především pro pochopení zadaného problému (například pro vygenerování grafické ukázky ZIGZAG algoritmu). Dále pro zhodnocení vypracování a uvedení neřešených problémů a námětů na vylepšení.

4.8 Literatura

[1] ITU-T. *Recommendation T.81: Digital compression and coding of continuous-tone still images.* Ženeva: ITU, 1992. Dostupné z: `https://www.w3.org/Graphics/JPEG/itu-t81.pdf`

[2] GONZALEZ, R. C., WOODS, R. E. *Digital Image Processing.* 4. vyd. New York: Pearson, 2018.

[3] SVOBODA, S. Nové transformace pro formát JPEG. In: *Excel@FIT 2017.* Brno: FIT VUT v Brně, 2017. Dostupné z: `https://excel.fit.vutbr.cz/submissions/2017/009/9.pdf`

[4] BAYER, T. *Komprese a kompresní algoritmy.* Přednáška. Praha: ČVUT v Praze, Fakulta stavební.